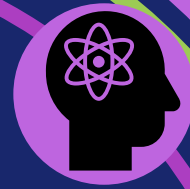
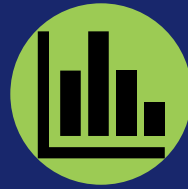
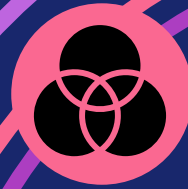


2026年度 前期 データマイニング特論 (I20教室)
第12回 2026年7月2日(木) 13:10~14:40



データマイニング特論

第12回

時系列データマイニング

本科目シラバスの確認 3（授業計画）

#	日程	内容
第1回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第2回	4/23（木）	pandasによるデータ操作の基礎
第3回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第4回	5/7（木）	データの前処理と特徴量エンジニアリング
第5回	5/14（木）	分類の基礎 — k近傍法と決定木
第6回	5/21（木）	アンサンブル学習
第7回	5/28（木）	線形分類モデルと回帰
第8回	6/4（木）	モデル評価・選択と不均衡データ
第9回	6/11（木）	クラスタリング手法
第10回	6/18（木）	アソシエーション分析と異常検知
第11回	6/25（木）	テキストマイニング
第12回	7/2（木）	時系列データマイニング
第13回	7/9（木）	深層学習の概観と使いどころ
第14回	7/16（木）	公平性・倫理と再現性
第15回	7/23（木）	まとめ

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1



時系列データとは
順序が情報であることを理解しよう！

時系列データは身近にある

【分野を問わず、順序を持つデータはすべて時系列】



環境モニタリング

気温・水質・大気の
連続観測



遺伝子発現

時間経過にともなう
発現量の変化



感染・医療

感染者数・バイタル
サインの推移



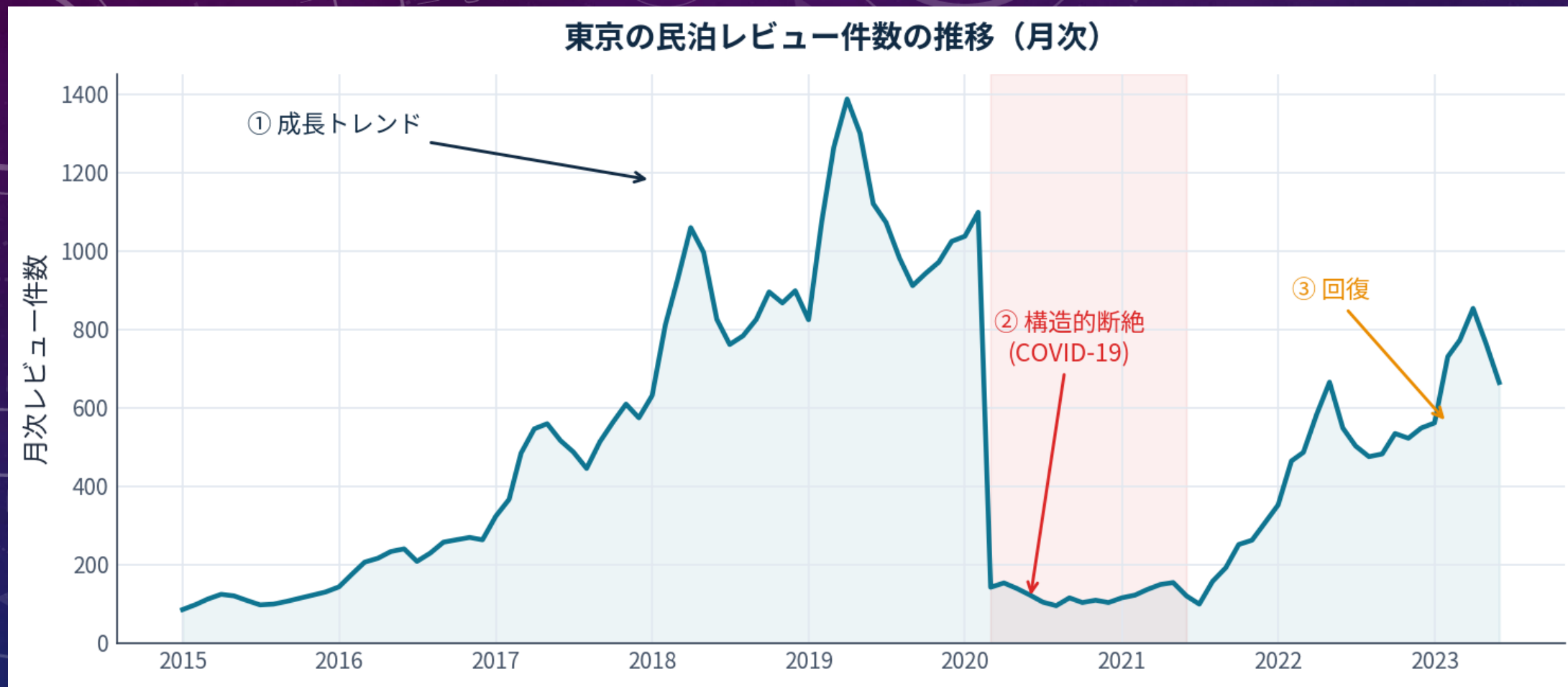
民泊・ビジネス

予約数・価格・
レビュー件数

先生方の研究データ（環境DNAの時系列、群集組成の季節変化 など）にも、そのまま応用できる。

走る例：東京の民泊レビュー件数

【Inside Airbnb Tokyo の reviews を月次集計（2015-01～2023-06、102か月）】



この1本に、今日の材料がすべて詰まっている。

①成長トレンド

②COVID による構造的断絶

③回復

2

分解、平滑化、DTW、
変化点、異常検知

5つの道具を目的に応じて選ぼう！



時系列マイニングの5つの仕事

【この回の地図】



① 分解

トレンド・季節性・残差
に分ける



② 平滑化

ノイズを落として本質を
見る



③ 類似性 (DTW)

形の近さを測り、分類する



④ 変化点・異常

切り替わり・外れを
見つける



⑤ 予測

(道具として軽く) 先を
見通す

The background is a gradient of dark blue and purple, speckled with small white dots. On the left side, there are several concentric circular patterns. One prominent circle has a degree scale around its perimeter, with labels from 140 to 260 in increments of 10. Other circles of varying sizes are scattered across the left and top-left areas, some with dashed lines and arrows indicating a clockwise direction. The Chinese characters '分解' are centered in the middle-right area, with a soft yellow glow around them.

分解

分解

【時系列を3つの成分に分ける】

観測値

実際に観測
された系列

=

トレンド

長期的な増減の
方向

+

季節性

周期的な繰り返し

+

残差

説明できない変動

加法モデル $y = T + S + R$

季節変動の幅が水準によらず一定のとき

乗法モデル $y = T \times S \times R$

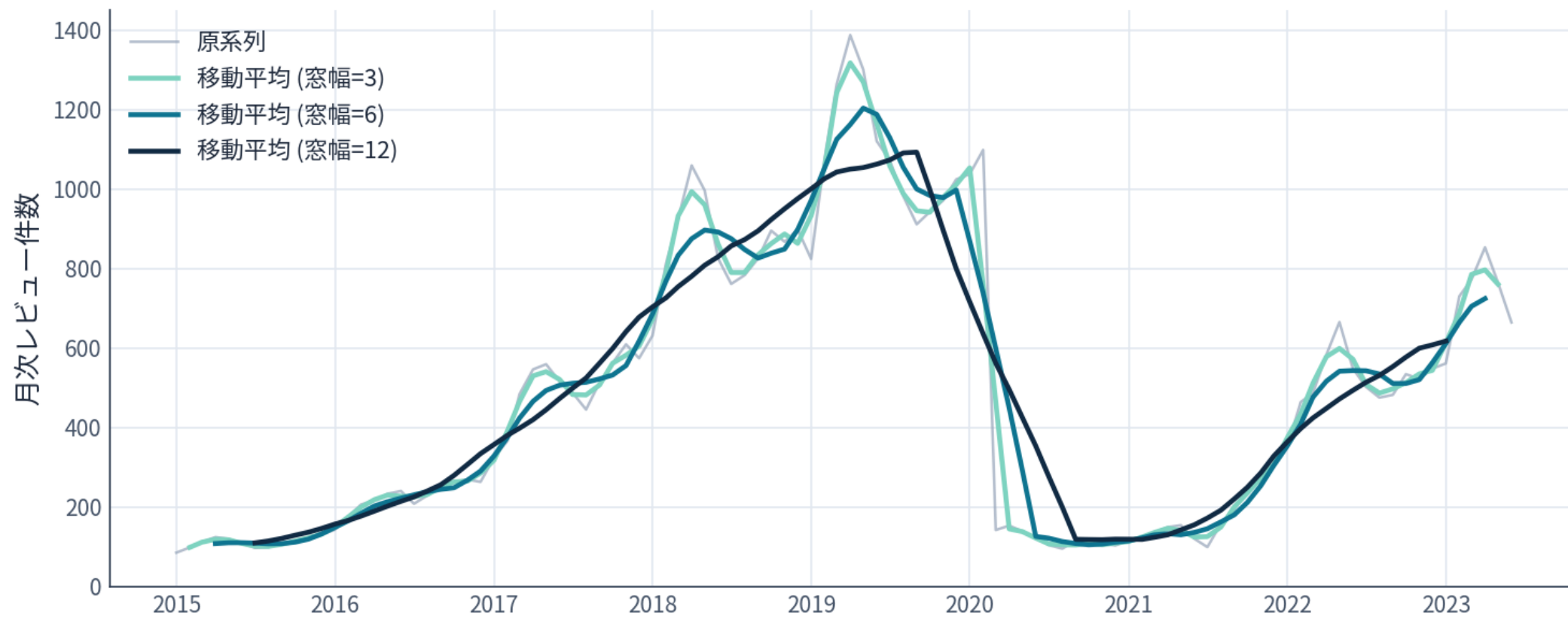
水準が上がると変動幅も拡大
(対数変換で加法に帰着)

今日のデータは乗法寄り — ピーク期ほど季節の振れ幅が大きい。
STL はこれを柔軟に扱える。

時系列データマイニング トレンド抽出

【移動平均】

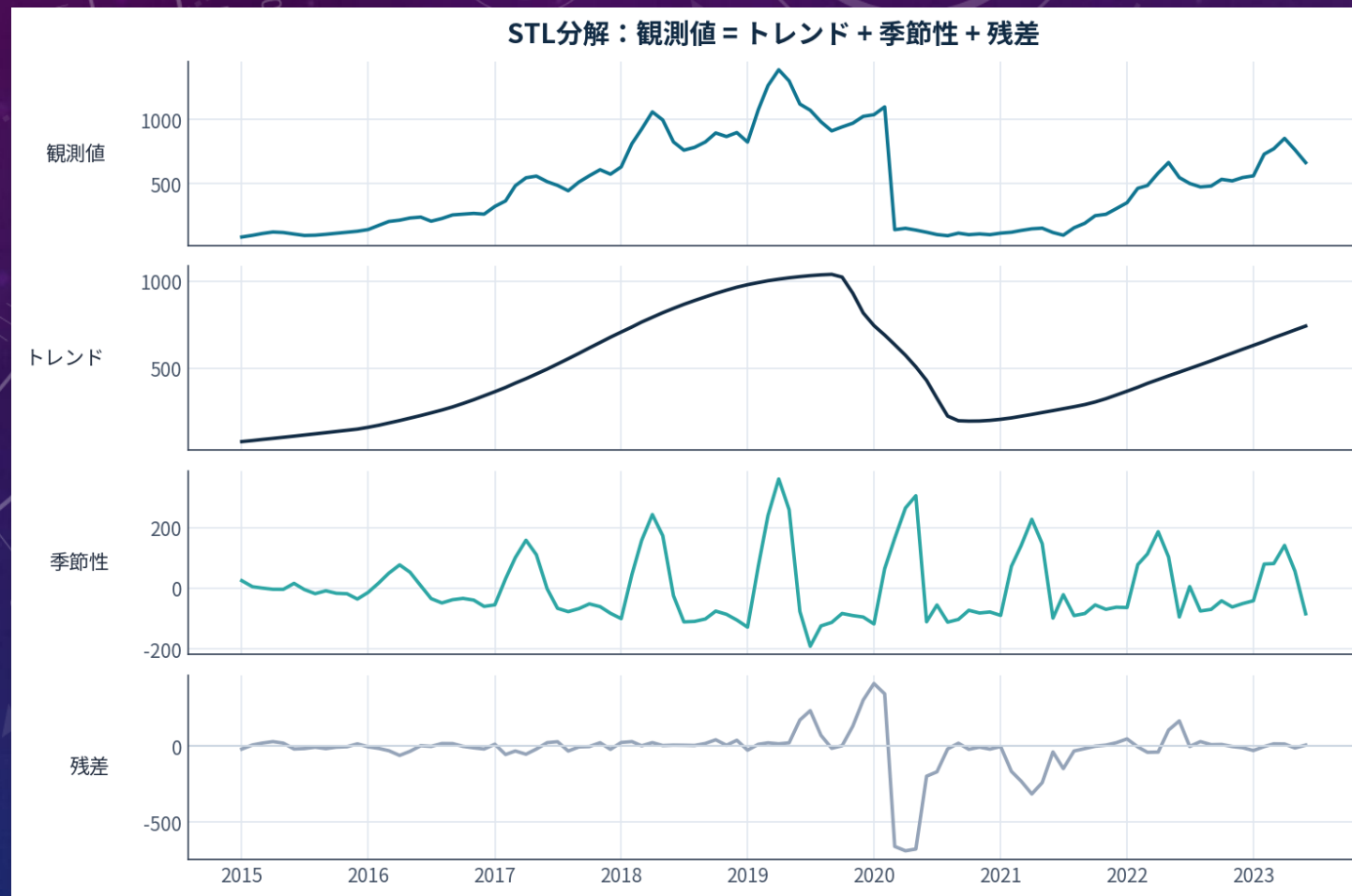
移動平均：窓幅を広げるほど滑らか（追従性は低下）



窓幅を広げるほど滑らかになるが、追従が遅れ、両端が欠ける。
トレンドを見るなら広め、変化を捉えるなら狭め。

STL分解

【観測値を3成分に自動分解】



STL = Seasonal-Trend decomposition using LOESS。
robust=True で外れ値に強い。季節ピークは春（3～5月）。

季節調整

【季節性を除くと本質が見える】

観測値

季節で上下して、
トレンドや異変が
見えにくい



観測値 - 季節成分

季節調整済み系列

トレンド・変化点・
異常が浮かび
上がる

ニュースの「季節調整済み失業率」と同じ発想。
変化点検出・異常検知の前処理としても効く。

本日の環境準備

いろいろ試してみよう！

日本語対応のモジュールをインストールする(その1)

```
!pip install japanize-matplotlib
```

日本語対応のモジュールをインストールする(その2)

```
!apt-get install -y fonts-ipafont-gothic
```

そのほか本日の必要なライブラリーをインストールする

```
!pip install statsmodels ruptures tslearn prophet
```

各ライブラリーをインポートする

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import japanize_matplotlib
```

データの読み込み (I)

いろいろ試してみよう！

データ読み込み

```
REVIEWS_URL = ("https://data.insideairbnb.com/japan/kant%C5%8D/tokyo/"
               "2023-06-29/visualisations/reviews.csv")

def load_reviews():
    """reviews を (listing_id, date) の long DataFrame で返す。失敗時は合成。"""
    try:
        df = pd.read_csv(REVIEWS_URL, parse_dates=["date"])
        df = df.rename(columns={c: c.strip() for c in df.columns})
        df = df[["listing_id", "date"]].dropna()
        print(f"[実データ] reviews を読み込みました: {len(df):,} 行 "
              f"({df['date'].min().date()} ~ {df['date'].max().date()})")
        return df, True
    except Exception as e:
        print(f"[フォールバック] 実データを取得できませんでした({type(e).__name__})."
              f"合成データで続行します。")
        return synthesize_reviews(), False
```

#次に続く。。。

データの読み込み (2)

いろいろ試してみよう！

```
def synthesize_reviews(n_listings=120):
    """成長＋年周期＋COVID断絶＋回復の構造を持つ reviews 風 long DataFrame を生成。
    物件ごとに3つの利用パターン(平日安定/観光シーズン/連休集中)を割り当てる。"""
    months = pd.date_range("2015-01-01", "2023-06-01", freq="MS")
    t = np.arange(len(months))
    # 全体の水準(1物件あたり月次レビュー期待値のベース)
    trend = 0.05 + 1.6 / (1 + np.exp(-(t - 34) / 9.0))
    covid = np.ones(len(months))
    for i, d in enumerate(months):
        if pd.Timestamp("2020-03-01") <= d < pd.Timestamp("2021-07-01"):
            covid[i] = 0.10
        elif d >= pd.Timestamp("2021-07-01"):
            ms = (d.year - 2021) * 12 + (d.month - 7)
            covid[i] = 0.10 + 0.55 * (1 - np.exp(-ms / 12.0))
    mth = months.month.to_numpy()

    def seasonal_profile(kind):
        if kind == "weekday": # 平日安定型:年間ほぼ一定
            return 1.0 + 0.10 * np.cos(2 * np.pi * (mth - 4) / 12)
        if kind == "season": # 観光シーズン型:春と秋にピーク
            return (0.6 + 1.1 * np.exp(-((mth - 4) ** 2) / 3.0) + 1.0 * np.exp(-((mth - 10) ** 2) / 3.0))
        # holiday:連休(GW=5月, お盆=8月, 年末年始=12/1月)に突出
        return (0.5 + 1.3 * np.exp(-((mth - 5) ** 2) / 0.8) + 1.2 * np.exp(-((mth - 8) ** 2) / 0.8)
                + 1.4 * np.exp(-((mth - 1) ** 2) / 0.8) + 1.4 * np.exp(-((mth - 12) ** 2) / 0.8))
```

#次に続く。。。

データの読み込み (3)

いろいろ試してみよう！

```
rows = []
kinds = ["weekday", "season", "holiday"]
for lid in range(1, n_listings + 1):
    kind = kinds[lid % 3]
    scale = RNG.uniform(0.6, 1.6)
    lam = np.clip(trend * covid * seasonal_profile(kind) * scale, 0, None)
    counts = RNG.poisson(lam)
    for i, c in enumerate(counts):
        if c <= 0:
            continue
        # その月内のランダムな日にレビューを散らす
        days = RNG.integers(1, 28, size=c)
        for dd in days:
            rows.append((lid, pd.Timestamp(months[i].year, months[i].month, int(dd))))
df = pd.DataFrame(rows, columns=["listing_id", "date"])
return df

reviews, IS_REAL = load_reviews()
```


月次集計（全体のレビュー件数の時系列）

いろいろ試してみよう！

```
monthly = (reviews.set_index("date")
            .resample("MS").size()
            .rename("reviews"))
# 端の欠測を避けるため、観測が薄い最初期は軽く除外(実データ対策)
monthly = monthly[monthly.index >= "2015-01-01"]
print(f"¥n月次系列: {len(monthly)} か月  "
      f"最大 {int(monthly.max())}({monthly.idxmax().date()}) / "
      f"最小 {int(monthly.min())}({monthly.idxmin().date()})")

plt.figure(figsize=(11, 4))
plt.plot(monthly.index, monthly.values, color="#0E7490", lw=2)
plt.title("東京の民泊レビュー件数(月次)" + (" " if IS_REAL else " ※合成データ"))
plt.ylabel("件数"); plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```



STL分解（トレンド・季節性・残差）①

いろいろ試してみよう！

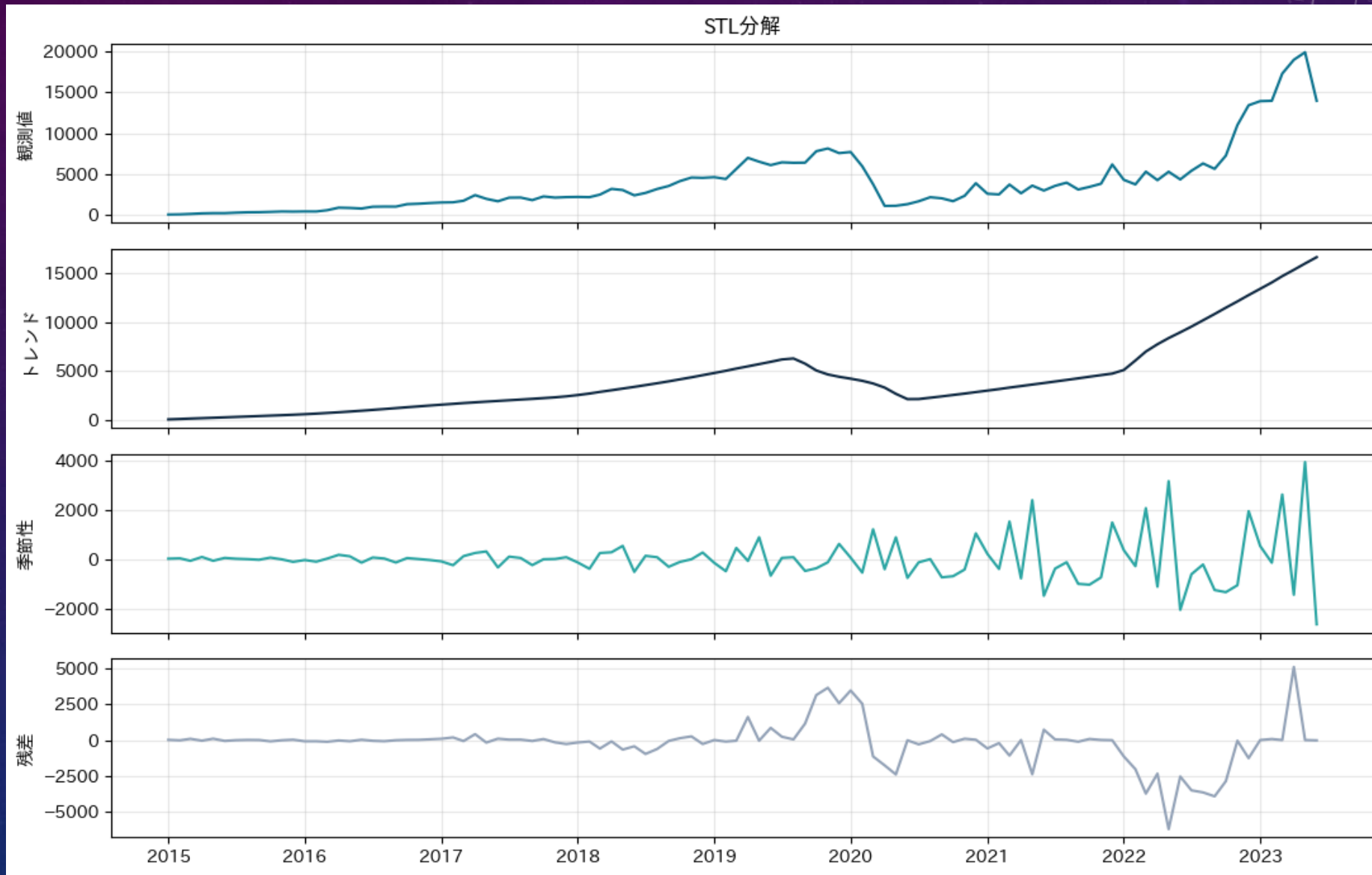
```
from statsmodels.tsa.seasonal import STL

ser = monthly.astype(float)
stl = STL(ser, period=12, robust=True).fit()

fig, ax = plt.subplots(4, 1, figsize=(11, 7), sharex=True)
for a, (y, lab, c) in zip(ax, [
    (ser.values, "観測値", "#0E7490"),
    (stl.trend.values, "トレンド", "#102A43"),
    (stl.seasonal.values, "季節性", "#2CA6A4"),
    (stl.resid.values, "残差", "#94A3B8")]):
    a.plot(ser.index, y, color=c); a.set_ylabel(lab); a.grid(alpha=.3)
ax[0].set_title("STL分解"); plt.tight_layout(); plt.show()

# 季節性がプラスに大きい月(=繁忙期)
seas_prof = pd.Series(stl.seasonal.values, index=ser.index.month).groupby(level=0).mean()
print("繁忙期(季節成分が大きい月)top3:", list(seas_prof.sort_values(ascending=False).head(3).index))
```

STL分解（トレンド・季節性・残差）②



平滑化

The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on the left side is a large, semi-circular degree scale ranging from 140 to 260. Several concentric circles and arcs are scattered across the image, some with arrows indicating a clockwise direction. The text '平滑化' is centered in the middle-right area.

平滑化

【ノイズを落として本質を残す】

窓を狭く

- 変化に敏感に反応する
- ノイズも拾いやすい
- 短期の異変を見たいとき

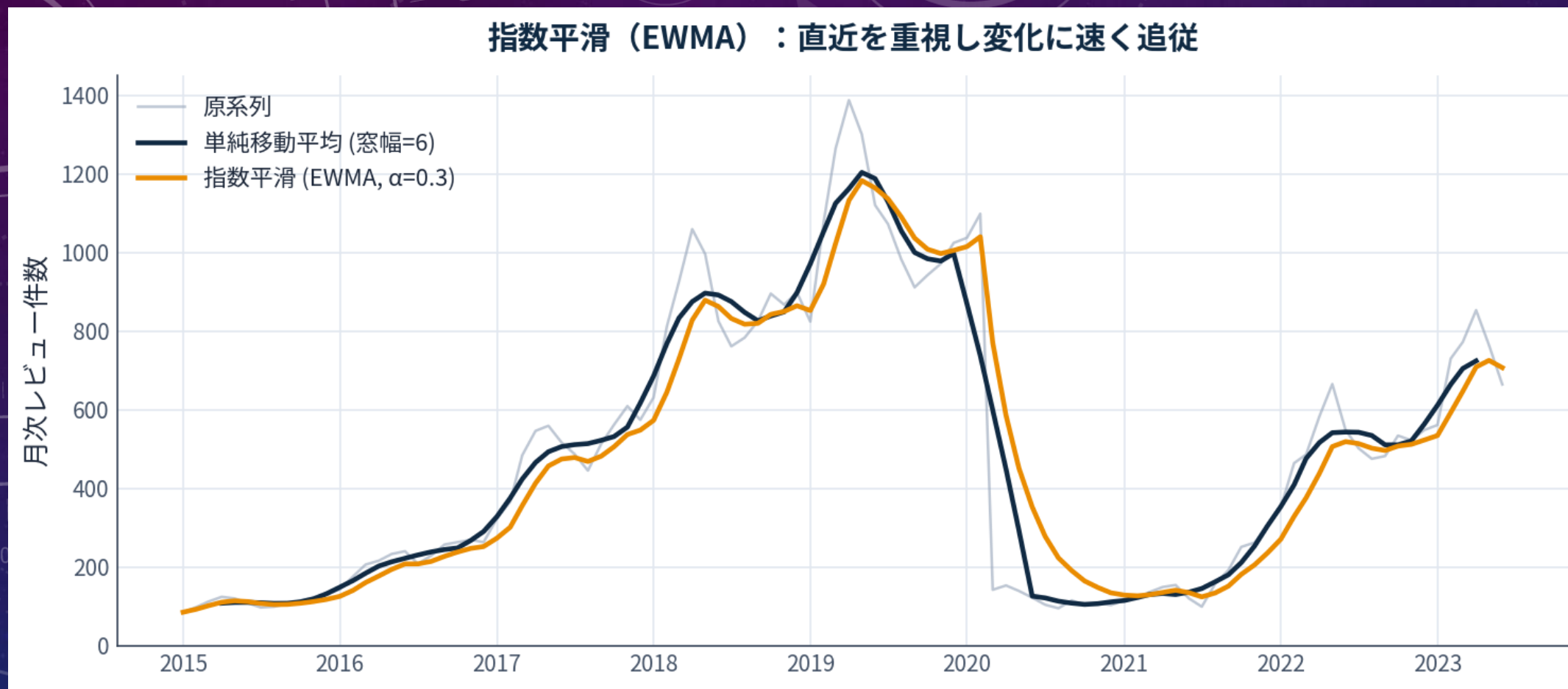
窓を広く

- 滑らかで安定する
- 追従が鈍く、端が欠ける
- 長期トレンドを見たいとき

単純移動平均は全点を等しく扱う。
「直近ほど重視したい」なら加重移動平均。
その代表がEWMA。

指数平滑 (EWMA)

【直近を重視する】



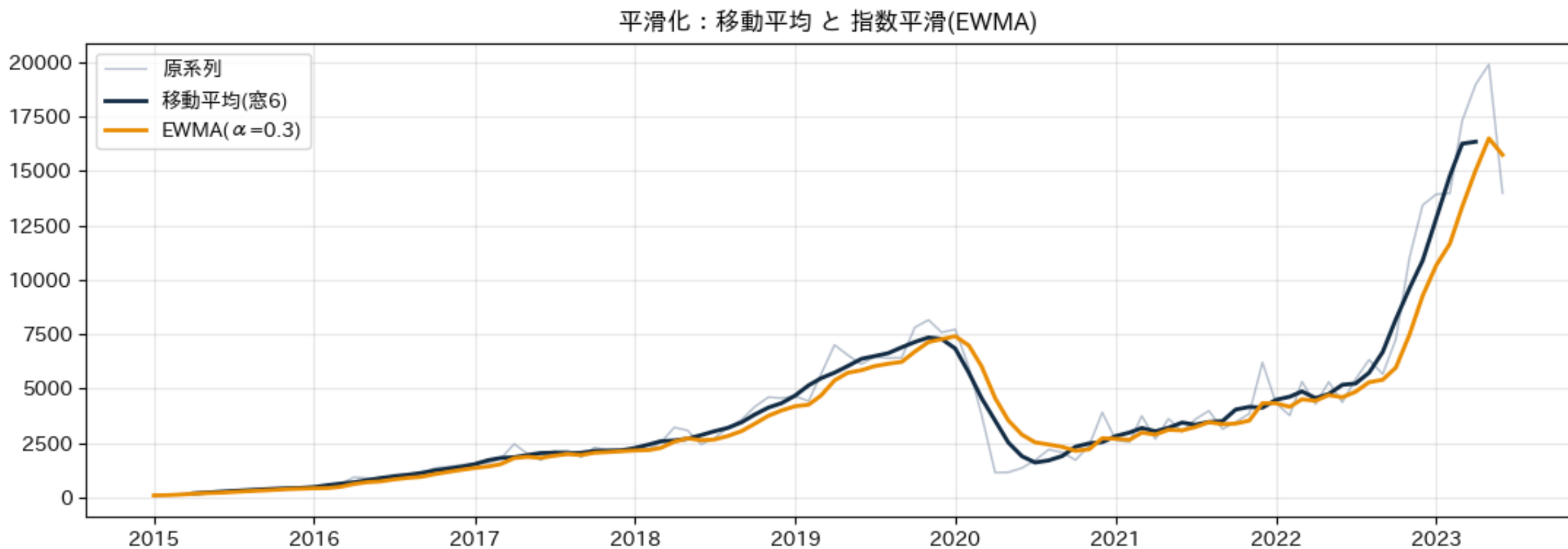
古いデータの重みを指数的に減衰。

α が大きいほど直近重視で追従が速い。両端が欠けないのも実務上の利点。

平滑化：移動平均 と EWMA

いろいろ試してみよう！

```
plt.figure(figsize=(11, 4))
plt.plot(ser.index, ser.values, color="#94A3B8", lw=1, alpha=.7, label="原系列")
plt.plot(ser.index, ser.rolling(6, center=True).mean(), color="#102A43", lw=2, label="移動平均(窓6)")
plt.plot(ser.index, ser.ewm(alpha=0.3).mean(), color="#EA8C00", lw=2, label="EWMA( $\alpha=0.3$ )")
plt.legend(); plt.grid(alpha=.3); plt.title("平滑化:移動平均 と 指数平滑(EWMA)")
plt.tight_layout(); plt.show()
```



The background is a dark blue gradient with a starry texture. On the left side, there are several concentric circular patterns. One large circle has a degree scale from 140 to 260. Other smaller circles have arrows indicating clockwise or counter-clockwise rotation. The text '類似性 (DTW)' is centered in the middle of the image.

類似性 (DTW)

類似性とDTW（I）

【時系列の「近さ」をどう測るか】



問い：物件Aと物件Bの予約パターンは「似ている」か？

似た形の系列をまとめれば、物件を「タイプ分け」できる。まず「似ている」を数値で定義する必要がある。

素朴なアイデア：各時点の差を全部足す（ユークリッド距離）。

…しかし、これには落とし穴がある。次のスライドで確認する。

類似性とDTW（2）

【ユークリッド距離の限界】

1

点对点で固定比較

同じ時刻どうしを一對一で引き算する

2

位相ずれに弱い

形が同じでもタイミングが少しずれると「遠い」

3

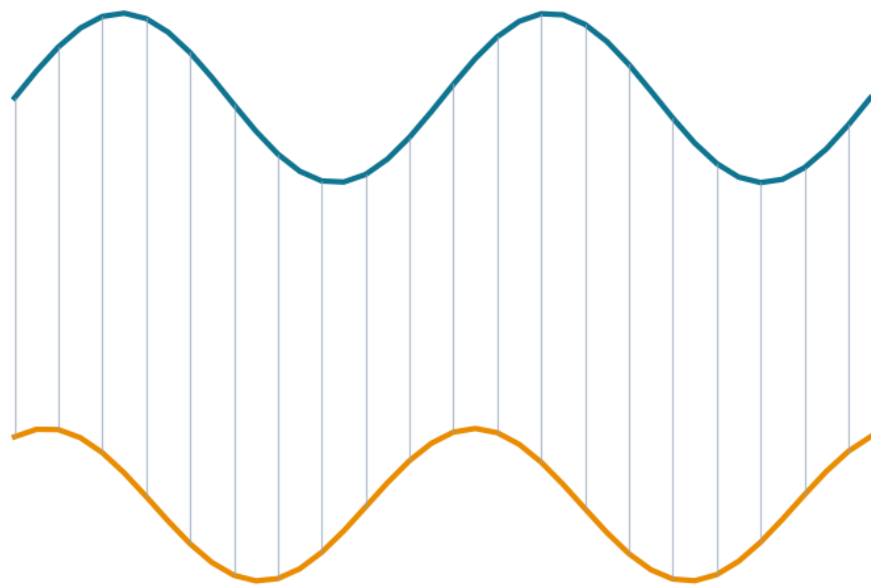
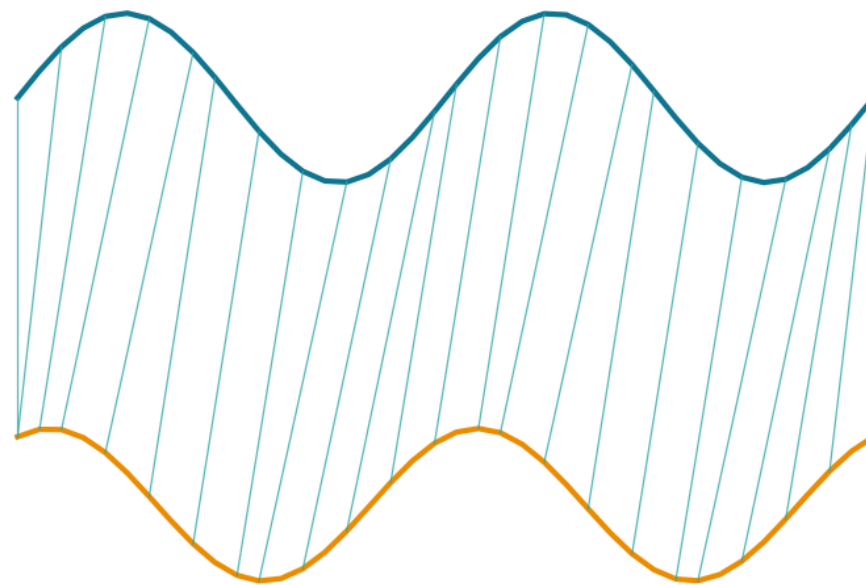
実データでは伸縮が普通

繁忙期の立ち上がりが年によって数週ずれる 等

解決策：時間軸を「伸縮」させて形どうしを対応づける。
これが、DTW（動的時間伸縮法）。

【時間を伸縮させて形を比較する】

位相がずれた波形：DTWは「形の近さ」を捉える

ユークリッド距離 = 26
(同じ時刻どうしを固定で比較)DTW距離 = 6
(時間を伸縮させて形を対応づけ)

同じ形の2波形でも ユークリッド距離 = 26 と大きい。
DTW は時間軸を伸縮して山と山・谷と谷を対応づけ、
DTW距離 = 6 と「近い」と正しく判定する。

DTW（動的時間伸縮法） — まず概念をミニ例で確認

いろいろ試してみよう！

```
def dtw_distance(x, y, return_path=False):
    """素朴なDTW( $O(NM)$ )。距離と、必要ならワーピングパスを返す。"""
    N, M = len(x), len(y)
    D = np.full((N + 1, M + 1), np.inf); D[0, 0] = 0.0
    for i in range(1, N + 1):
        for j in range(1, M + 1):
            cost = abs(x[i - 1] - y[j - 1])
            D[i, j] = cost + min(D[i - 1, j], D[i, j - 1], D[i - 1, j - 1])
    if not return_path:
        return D[N, M]
    path = [(N - 1, M - 1)]; i, j = N, M
    while i > 1 or j > 1:
        _, i, j = min([(D[i-1, j-1], i-1, j-1), (D[i-1, j], i-1, j), (D[i, j-1], i, j-1)],
                      key=lambda z: z[0])
        path.append((i - 1, j - 1))
    return D[N, M], path[::-1]
```

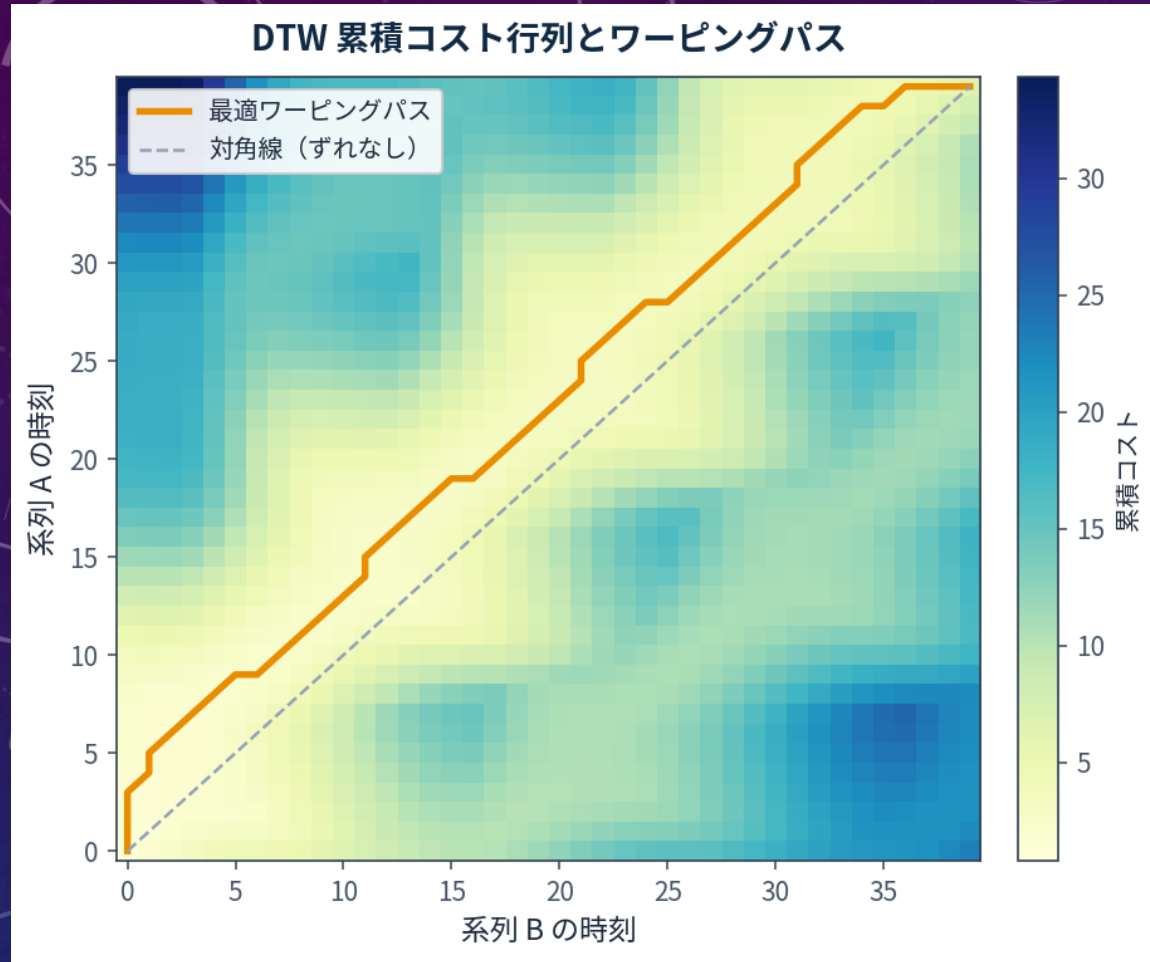
位相のずれた2波形: ユークリッド vs DTW

```
tt = np.linspace(0, 1, 40)
a = np.sin(2 * np.pi * 2 * tt)
b = np.sin(2 * np.pi * 2 * tt + 1.1) * 0.9
eucl = np.abs(a - b).sum()
dtw_d = dtw_distance(a, b)
print(f"¥n[DTWミニ例] ユークリッド距離 = {eucl:.1f} / DTW距離 = {dtw_d:.1f}"
      f" → DTWの方が「形が同じ」を正しく近いと判定")
```

[DTWミニ例] ユークリッド距離 = 25.6 / DTW距離 = 6.5 → DTWの方が「形が同じ」を正しく近いと判定

DTWの仕組み

【コスト行列とワーピングパス】



① 総当たりのコスト表を作る

2系列の全時点ペアの距離を格子に。

② 最小コストの経路を探す

左下→右上へ、動的計画法で累積最小の道（ワーピングパス）を求める。

③ 経路の総コスト = DTW距離

対角線に近いほど「ずれ」が小さい。

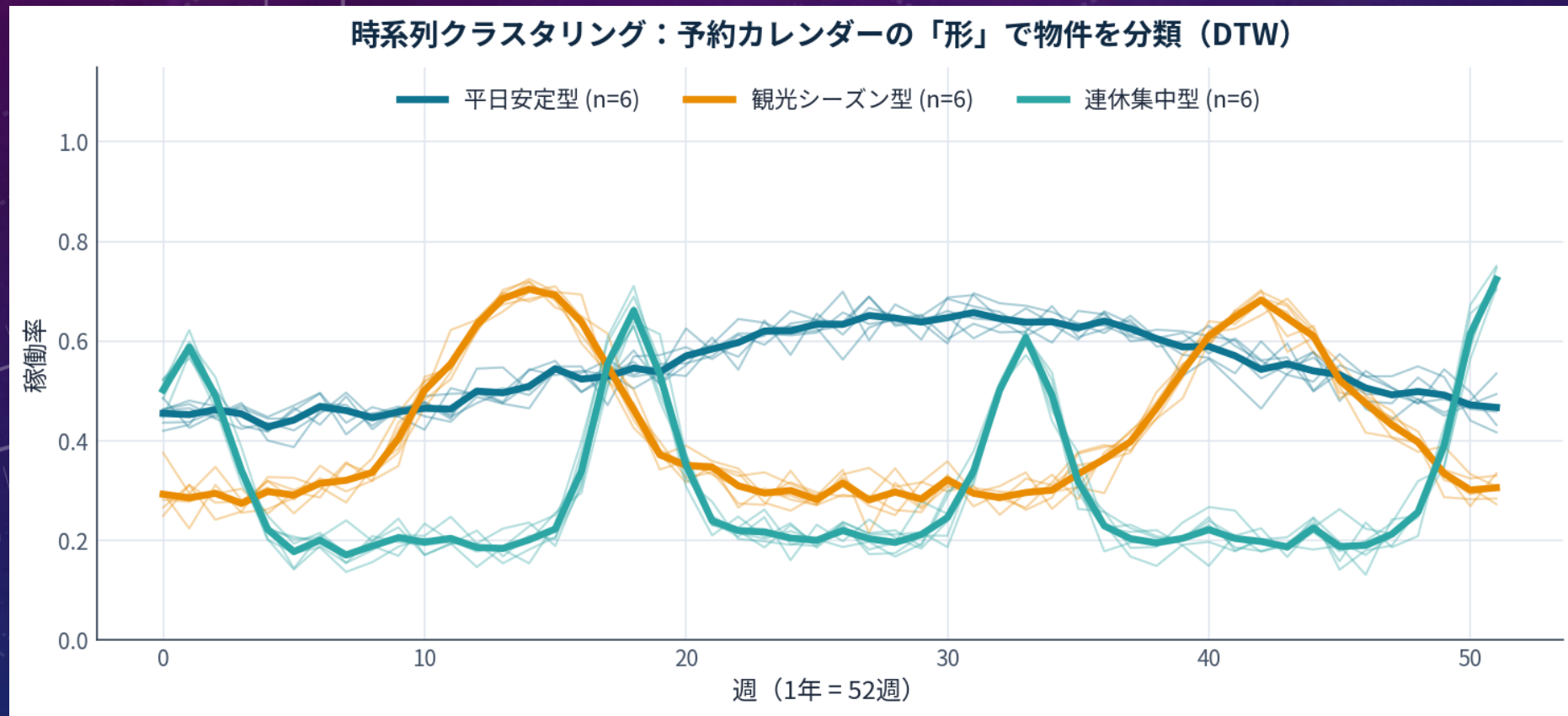
計算量は $O(N \times M)$ 。

長い系列どうしても重くなる点に注意（バンド制約などで高速化）。

DTWの応用

【時系列クラスタリング】

DTW距離 + 階層的クラスタリング（第9回の手法が時系列にも効く）



予約カレンダーの「形」で物件进行分类。

3タイプにきれいに分離：平日安定型・観光シーズン型・連休集中型（各6件）。

```
from scipy.cluster.hierarchy import linkage, fcluster
from scipy.spatial.distance import squareform

# レビュー数の多い上位物件を選び、共通期間の月次系列を作る
top_ids = reviews["listing_id"].value_counts().head(30).index
common = pd.date_range(monthly.index.min(), monthly.index.max(), freq="MS")

def listing_series(lid):
    s = (reviews[reviews.listing_id == lid].set_index("date")
         .resample("MS").size().reindex(common, fill_value=0).astype(float))
    # 形を比較したいので各系列を正規化(合計1)
    tot = s.sum()
    return (s / tot).values if tot > 0 else s.values

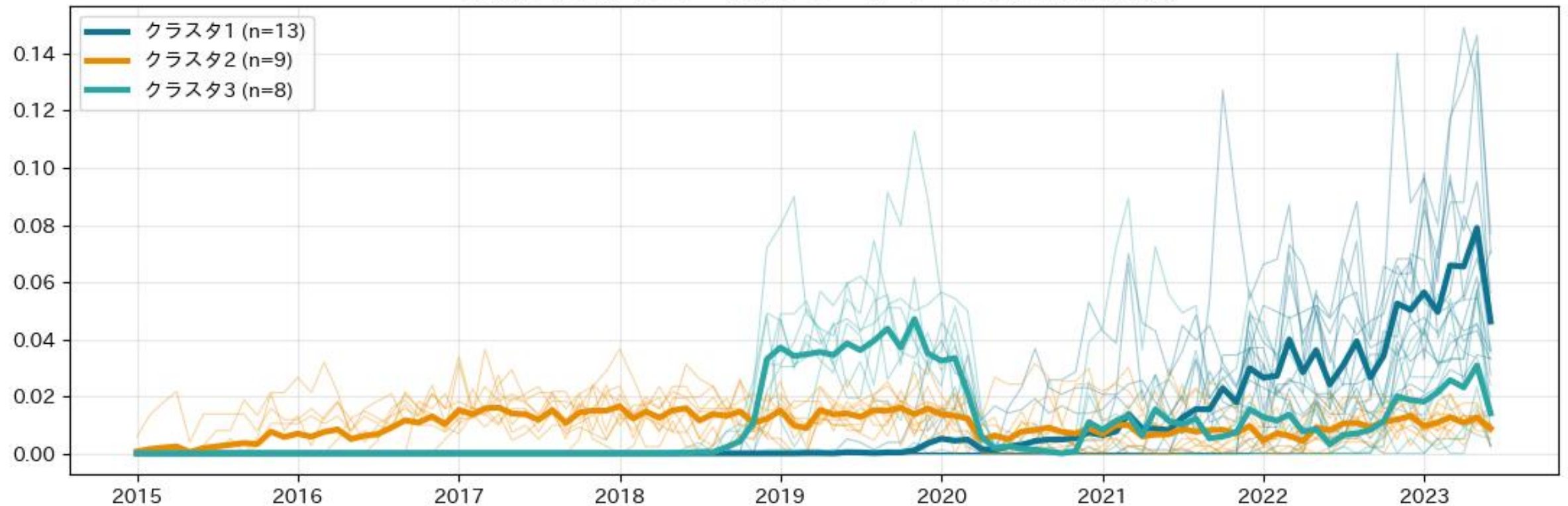
curves, ids = [], []
for lid in top_ids:
    c = listing_series(lid)
    if np.count_nonzero(c) >= 6:          # 観測が薄すぎる物件は除外
        curves.append(c); ids.append(lid)
curves = np.array(curves)
K = min(3, len(curves))

#次に続く。。。
```

```
if len(curves) >= K and K >= 2:
    n = len(curves)
    Dm = np.zeros((n, n))
    for i in range(n):
        for j in range(i + 1, n):
            Dm[i, j] = Dm[j, i] = dtw_distance(curves[i], curves[j])
    Z = linkage(squareform(Dm), method="average")
    labels = fcluster(Z, t=K, criterion="maxclust")
    print(f"¥n[クラスタリング] {n}物件を DTW+階層的クラスタリングで {K}群に分割: "
          f"{ {int(k): int((labels==k).sum()) for k in np.unique(labels)} }")

    colors = ["#0E7490", "#EA8C00", "#2CA6A4", "#A855F7"]
    plt.figure(figsize=(11, 4))
    for i in range(n):
        plt.plot(common, curves[i], color=colors[(labels[i]-1) % len(colors)], lw=.8, alpha=.35)
    for k in np.unique(labels):
        plt.plot(common, curves[labels == k].mean(axis=0),
                 color=colors[(k-1) % len(colors)], lw=3, label=f"クラスタ{k} (n={int((labels==k).sum())})")
    plt.legend(); plt.grid(alpha=.3)
    plt.title("時系列クラスタリング: 予約(レビュー)パターンの形で物件を分類")
    plt.tight_layout(); plt.show()
else:
    print("¥n[クラスタリング] 対象物件が不足したためスキップしました。")
```

時系列クラスタリング：予約(レビュー)パターンの形で物件进行分类



The background is a deep blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. A large circle on the left features a scale from 140 to 260 in increments of 10. Other smaller circles and arcs are scattered across the frame, some with arrows indicating a clockwise direction. The text '变化点・異常' is centered in the middle of the image.

变化点・異常

変化点検出

【レジームの切り替わり】



変化点 = 系列の統計的性質（平均・分散）が
切り替わる時点

設備の故障、政策の変更、市場の転換、生態系のレジームシフト。
「いつ何かが変わったか」を客観的に特定する。

手法：PELT

動的計画法で「最適な分割」を高速に
探索（Pruned Exact Linear Time）。

コツ：先に季節性を除く

季節変動を変化点と誤検出しないよう
「分解 → 季節性除去 → 検出」の順で行う。

変化点検出の実践

【季節性を除いた系列に PELT を適用】



3つの変化点（2017-07 / 2020-06 / 2022-02）を自動検出
→ 4レジーム：急成長→高原→COVID崩落→回復。

※ 変化点は急落の瞬間そのものより、新しい水準が安定した時点（2020-06）に置かれることがある。

```
import ruptures as rpt

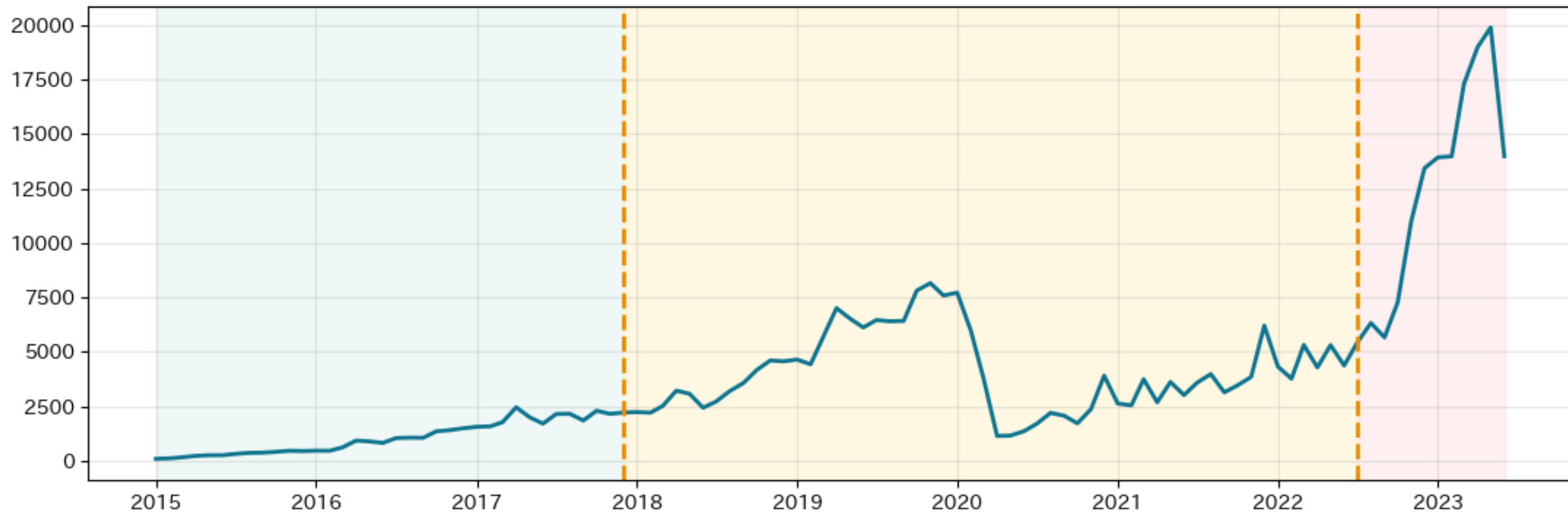
deseason = (ser - stl.seasonal).values          # 季節性を除く
algo = rpt.Pelt(model="rbf", min_size=4).fit(deseason)
bkps = [b for b in algo.predict(pen=6) if b < len(ser)]
cp_dates = [ser.index[b].strftime("%Y-%m") for b in bkps]
print(f"¥n[変化点検出] 検出された変化点: {cp_dates}")
print(" ※ 変化点は急落の瞬間そのものより、"
      "新しい水準が安定した時点に置かれることがある点に注意。")

plt.figure(figsize=(11, 4))
prev = 0
seg_colors = ["#E0F2F1", "#FEF3C7", "#FEE2E2", "#E0F2F1", "#F0FDDFA"]
for k, b in enumerate(bkps + [len(ser)]):
    plt.axvspan(ser.index[prev], ser.index[min(b, len(ser)-1)],
                color=seg_colors[k % len(seg_colors)], alpha=.5)
    prev = b
plt.plot(ser.index, ser.values, color="#0E7490", lw=2)
for b in bkps:
    plt.axvline(ser.index[b], color="#EA8C00", ls="--", lw=2)
plt.title("変化点検出(PELT):レジームの切り替わり")
plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```


時系列データマイニング

変化点検出 (PELT) — 季節性を除いてから検出するのがコツ②

変化点検出(PELT)：レジームの切り替わり



時系列の異常は3種類



点異常

単独で大きく外れる

例) 1点だけ突出した予約数



文脈的異常

値は正常でも文脈で異常

例) 真夏なのに真冬の予約水準



集団異常

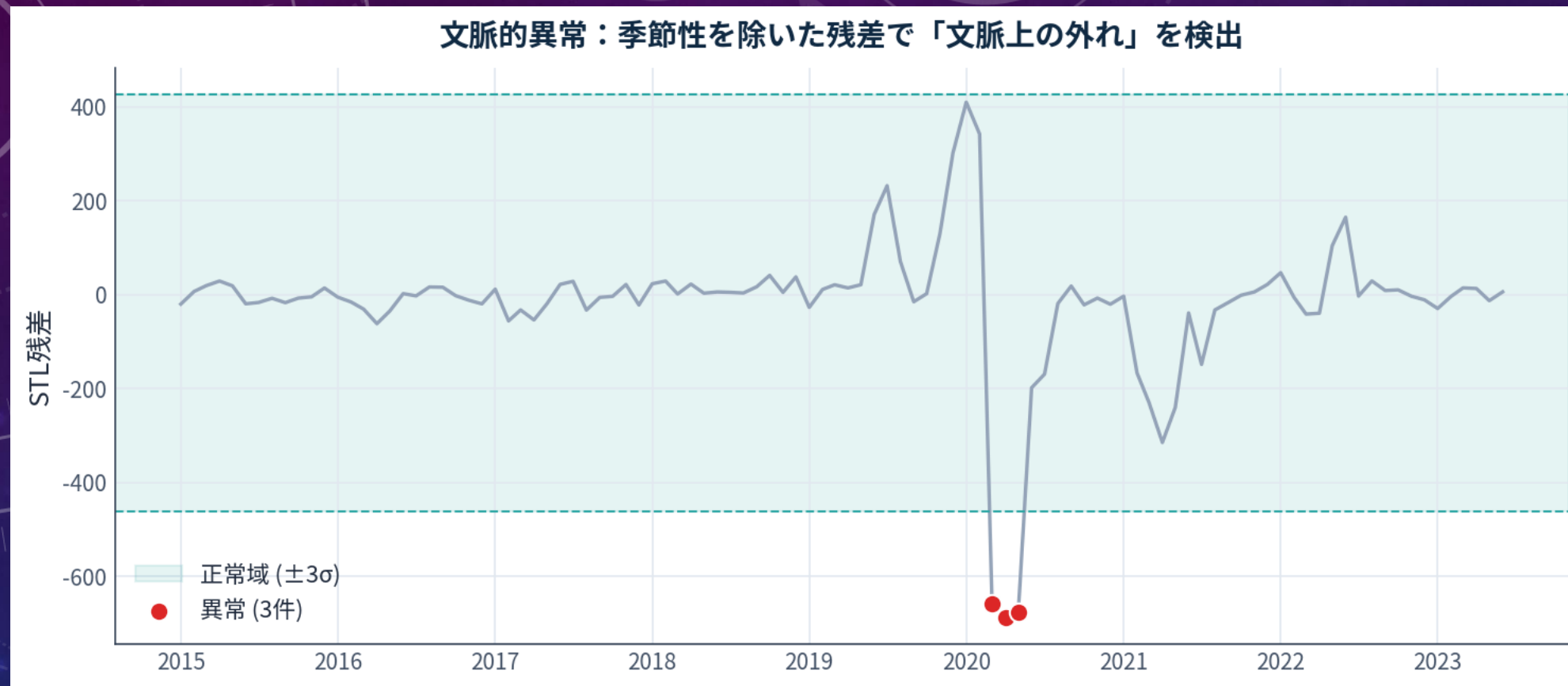
連続すると異常

例) 動くはずの期間の平坦化

時系列で特に重要なのは「文脈的異常」。
季節性という文脈を除いてから外れを見るのがポイント。

文脈的異常の実践

【季節性を除いた残差 $\pm 3\sigma$ で検出】



COVID 直撃の3か月（2020-03・04・05）を検出。

第10回の Isolation Forest / LOF は「多変量の空間的な外れ」を捉えた。
時系列では「季節性という文脈」を除いてから残差の外れを見る。

文脈的異常（季節性を除いた残差の $\pm 3\sigma$ ）①

いろいろ試してみよう！

```
resid = stl.resid
mu, sd = resid.mean(), resid.std()
thr = 3.0
mask = (resid - mu).abs() > thr * sd
print(f"¥n[文脈的異常] 残差  $\pm\{thr:.0f\}\sigma$  で {int(mask.sum())} 件検出: "
      f"{[d.strftime('%Y-%m') for d in ser.index[mask.values]]}")

plt.figure(figsize=(11, 4))
plt.plot(ser.index, resid.values, color="#94A3B8", lw=1.5)
plt.axhspan(mu - thr*sd, mu + thr*sd, color="#2CA6A4", alpha=.12, label=f"正常域  $\pm\{thr:.0f\}\sigma$ ")
plt.scatter(ser.index[mask.values], resid.values[mask.values],
            color="#DC2626", s=60, zorder=5, label="異常", edgecolor="white")
plt.legend(); plt.grid(alpha=.3)
plt.title("文脈的異常: 季節性を除いた残差で外れを検出")
plt.ylabel("STL残差"); plt.tight_layout(); plt.show()
```


文脈的異常（季節性を除いた残差の $\pm 3\sigma$ ）②

The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several white circular elements. A large circular scale on the left side features degree markings from 140 to 260 in increments of 10. Other circular patterns include concentric circles with arrows indicating clockwise or counter-clockwise movement, and dashed circular paths. The overall aesthetic is technical and futuristic.

予測

時系列データマイニング 予測は道具として

【Prophet】



Prophet 等は「トレンド+季節性+変化点」を自動分解して予測。
今日 手作業でやったことの総まとめ。

The background is a gradient of dark blue and purple, speckled with small white dots. On the left side, there are several concentric circular patterns. One large circle has a degree scale from 140 to 260. Other smaller circles have arrows indicating clockwise or counter-clockwise rotation. The text 'まとめ' is centered in the middle of the image.

まとめ

まとめ

【時系列マイニングのベストプラクティス 5か条】

- 1 まず可視化する** 生の系列を必ず目で見てから手法を選ぶ
- 2 順序を壊さない** ランダム分割は禁物 — 時系列分割を使う
- 3 分解してから考える** トレンド・季節性・残差に切り分ける
- 4 季節性を除いてから外れを見る** 変化点・異常は季節調整後に検出する
- 5 予測とパターン発見を混同しない** 目的に合った道具を選ぶ

3

本日の課題

頑張ってください。



課題

【課題】 期限：2026/7/8（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第12回（7/2）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力

```
!jupyter nbconvert --to html "aaa.ipynb"
```


【ADSセミナー】

本学の「先端データ科学研究センター」が主催するセミナーが6/25（木）から4週にわたって開催されます。

内容が盛りだくさんなので、**研究室を選択する前の参考になる**かと思います。

事前登録フォームから申し込みあり。

【Point】 普段の授業では聞けない話もあるかも！？

ADSセミナー

対象 東京情報大学
全学部・学年/大学院生/教職員

会場 1号館 201教室

【第1回】 6/25（木） 16:30 - 18:00

講師： 釣井 達也 先生
総合情報学科 准教授（数理情報研究ユニット）



量子ウォークに潜む群構造について

量子ウォークはランダムウォークの量子版と解釈される数学モデルであり、その振る舞いはランダムウォークと大きく異なります。本セミナーでは、量子ウォークの簡単な具体例を通してランダムウォークとの違いを観察します。特に、量子ウォークに潜む群構造に着目し、この群構造を活用して、量子ウォークの特徴的な挙動のひとつである「周期」を求める方法について紹介します。

【第2回】 7/2（木） 16:30 - 18:00

講師： 早稲田 篤志 先生
総合情報学科 准教授（情報セキュリティ研究ユニット）



個人情報保護と機械学習

プライバシーに関する情報は解析して使用することで有益な情報を得られることが期待できます。一方、漏洩等で外部に漏れると大きな問題となることがあります。そこで、基本的なプライバシー保護の指標を紹介し、機械学習との関係を考察します。

【第3回】 7/9（木） 16:30 - 18:00

講師： 西村 明 先生
総合情報学科 教授（情報メディア学系）



音響法科学（オーディオ・フォレンジクス）

音響法科学は、音響学の原理と技術を、犯罪捜査・法廷および不正等の検知のために活かす研究と学問を指します。話者識別、雑音除去と音声強調、録音属性推定に大別され、本講演では、前二者を概観したのち、AI音声の判別を含む後者の研究について概説します。

【第4回】 7/16（木） 16:30 - 18:00

講師： 小早川 睦貴 先生
総合情報学科 准教授（生命情報研究ユニット）



身体情報から読みとる心の働き

心理学的研究で心を読み取る際には、さまざまな人間の「反応」を測ることが不可欠です。一般的には反応時間や正答率、表情など目に見える反応を使いますが、発汗や視線など、目に見えない反応などを使って心を読む心理学的研究を紹介します。

<参加登録>

右のQRコードを読み取り、参加登録フォームへアクセスの上、必要事項を記入してお申し込みください。



受付期間： 5/26（火）～ 7/16（木）

<https://forms.office.com/r/JRmJe3qNmh>

早期研究体験プロジェクト
（対象：学部1・2年次）

本プログラムに興味のある学生は、先端データ科学研究センターまでお問い合わせください。

随時
募集中

<お問い合わせ>

東京情報大学 総合情報学部
先端データ科学研究センター
ads@rsch.tuis.ac.jp

